

TalentFlow — Project Deep Dive

Architecture, implementation decisions, debugging and engineering lessons

1. Product problem

Recruitment involves multiple related records and decisions: companies create roles, candidates apply, recruiters review applicants, and applications move through controlled stages. TalentFlow centralizes that workflow instead of treating jobs and applications as disconnected CRUD records.

2. Domain model

The core relationship is User → CandidateProfile / RecruiterProfile → Company → Job → Application. Supporting entities include Skill, JobSkill, StatusHistory, Interview, InterviewFeedback, Offer, SavedJob, Notification and AuditLog in the broader design.

3. API design

The API is versioned under `/api/v1/`. Jobs and applications use DRF generic views. The Job serializer exposes nested read-only skills plus write-only `skill_ids`. The application endpoint supports status updates, and status changes create StatusHistory records and candidate notifications.

4. Performance decisions

The job list uses `select_related` for company/recruiter and `prefetch_related` for skills. Application lists similarly optimize related job/candidate data and status history. This keeps the MVP simple while avoiding obvious N+1 query patterns.

5. Authentication and roles

The frontend sends DRF Token authentication. Job visibility is role-aware: recruiters see their own jobs while candidates see published jobs. Job creation associates the authenticated recruiter and their RecruiterProfile company server-side.

6. Feature debugging journey

A stale token caused authenticated requests to fail after token rotation; clearing local storage and signing in again restored access. The candidate demo password did not match the expected value, so it was reset through Django shell. The Pipeline initially displayed statuses but did not allow movement; adding a status selector and PATCH flow turned it into an interactive workflow. Skill IDs were also tested end-to-end to verify JobSkill associations.

7. Engineering approach

The project was developed incrementally: validate the backend first, verify API behavior, connect the React UI, test with real demo users, then polish the interface. Small changes were preferred over large rewrites so each feature could be verified before moving on.

8. What is deliberately not claimed

The MVP documentation does not claim automated test coverage, active Celery/Redis processing, AI matching, or a production deployment unless those features are actually enabled later. The architecture and blueprint describe the

intended progressive roadmap.